

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Department of Information Technology

Fall 2026-27

Operating Systems Lab BITE303P

PROJECT DESIGN & PROGRESS REPORT (REVIEW-I)

KernelLeakGuard: Detecting and Verifying Linux Kernel Memory Leaks

Submitted by

Om Gunjan Gupta 24BIT0316

Abhishek Kumar Singh 24BIT0309

Aarav Jain 24BIT0023

Under the Guidance of

Dr. ARIVUSELVAN K

Associate Professor, SCORE

VIT, Vellore.

Fall 2026-27

PROJECT DESIGN & PROGRESS REPORT (REVIEW-I)

1. Project at a Glance

KernelLeakGuard finds memory leaks on the error exits of Linux kernel code and proves them inside a real running kernel. In one sentence: **a static analyser points at the suspicious exit and the failing call that leads there; a virtual machine makes exactly that call fail; the kernel's leak detector, kmemleak, says whether memory was really lost.** This report describes the design, how the work is split, the algorithms, how far the implementation has come by Review-I, screenshots of the working prototype, and the problems we ran into.

Table 1.1: Summary of progress

Item	Status at Review-I
Problem, objectives, literature	Done (see the proposal)
Architecture and module design	Done
Static analyser	Working on the benchmark; refinements continue
Kernel, virtual machine, fault injection	Working
Runtime verification and verdicts	Working end to end
Benchmark modules with known answers	Done (32 modules)
Baselines, fix patches, dashboard, real modules	In progress for Review-II

2. System Architecture

The system has two halves that talk through files. On the **host** (Ubuntu in WSL2) the static analyser reads the compiled modules and writes a list of suspected leaks. In the **guest** (our own Linux 6.18 kernel inside QEMU) the agent loads each module and injects failures, and kmemleak reports what was lost. The Python orchestrator on the host connects the two: it prepares the guest's job, boots the virtual machine, reads the guest's logs back, and turns them into verdicts.

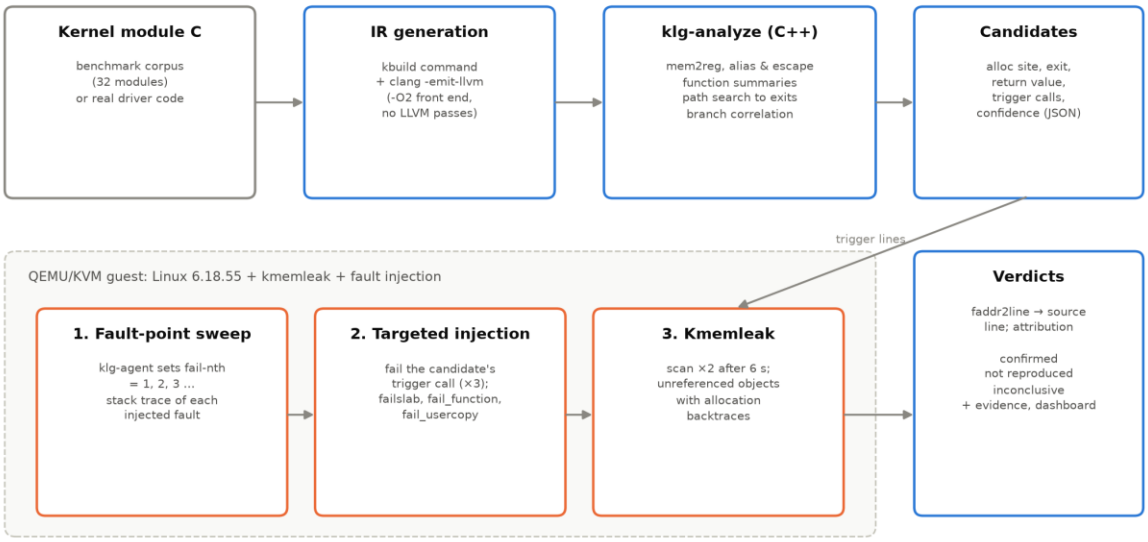


Figure 2.1: System architecture

The main pieces, in the order the data flows:

- **Build:** each module is compiled twice — normally (for the VM) and into LLVM IR (for the analyser), with exactly the kernel's own compiler flags.
- **klg-analyze (C++):** finds leak candidates and writes them as JSON.
- **klg (Python):** plans which failure to inject for each candidate and writes a shell script for the guest.
- **QEMU guest:** boots our kernel, runs the script, loads modules, injects failures, runs kmemleak, writes logs to a shared folder.
- **Verifier:** reads the logs, matches leaked objects to source lines and decides each verdict.
- **Reports:** command-line summary now; dashboard and documents for Review-II.

3. Module Design

We split the work into four modules with clear inputs and outputs, so each of us could work and test independently.

Table 3.1: Modules

Module	Owner	Input	Output
Static analyser (analyzer/)	Om Gunjan Gupta	LLVM IR of a module	candidates.json: allocation, leaking exit, failing call, error code
Kernel and VM (vm/, runtime/)	Abhishek Kumar Singh	Kernel source, modules, guest job script	Booted guest; agent logs; kmemleak reports

Verifier and orchestrator (klg/)	Aarav Jain	candidates.json, guest logs	Verdicts with evidence (verification.json)
Benchmark and evaluation (corpus/, klg/evaluate.py)	Aarav Jain	Modules with GT:LEAK / GT:SAFE marks	Precision, recall, comparison tables

3.1 Static analyser

Written in C++ on top of the LLVM 21 libraries. It first simplifies the IR (turns stack variables into plain values), then for each allocation call it collects every value that is a copy of the pointer (aliases), notes where the pointer is freed, returned or stored somewhere outside the function (ownership hand-over), and searches the paths to every return. Small summaries of helper functions (does this function free its argument? return new memory?) let it follow memory across function calls.

3.2 Kernel, virtual machine and agent

We build Linux 6.18.55 with clang and switch on kmemleak, the fault-injection framework (failslab, fail_function, fail_usercopy) and a few settings that keep kmemleak accurate. QEMU boots it with a tiny BusyBox file system and shares a host folder with it. The agent (`klg-agent`, written in C) loads a module or calls a test trigger while `/proc/self/fail-nth` makes the chosen allocation fail, and prints markers that the host can parse. A helper module, `klg_modwatch`, logs where modules are loaded so that stack traces can be turned back into source lines even after a module was unloaded.

3.3 Verifier

The Python verifier first runs a **mapping** pass (fail allocation 1, 2, 3, ... and record where each failure happened), then picks for each candidate the failure number that hits its trigger, runs those **confirmation** tests three times, and compares kmemleak's report with the candidate's allocation site.

3.4 Benchmark

32 small kernel modules written by us: 17 leaking allocation sites and 42 safe ones. Each allocation line carries a comment `/* GT:LEAK */` or `/* GT:SAFE */`, which is the ground truth for scoring. The safe modules contain the tricky patterns that fool naive tools: memory handed to a list, freed through a wrapper, freed in a loop, or reference counted.

3.5 Data formats

We do not need a database; all stages exchange JSON files, which are easy to inspect. A shortened candidate written by the analyser looks like this:

```
{
  "function": "sensor_setup",
  "allocator": "kmalloc",
  "alloc":  {"file": "klg_l02_goto_ladder.c", "line": 32},
  "exit":   {"file": "klg_l02_goto_ladder.c", "line": 54},
  "return_value": "-ENOMEM",
  "triggers": [{"callee": "kasprintf", "line": 38, "kind": "alloc"}],
  "confidence": "high",
  "category": "missing-release-on-error-path"
}
```

4. Algorithms and Flowcharts

4.1 Finding leaks (static analysis)

```
for each allocation call A in function F:
    aliases  = every value that is a copy of A's result
    events   = places where an alias is freed, returned, or stored outside F
    depth-first search over the paths from A to every return of F:
        skip the edge where A itself failed (A == NULL or IS_ERR(A))
        skip branches that contradict an earlier decision on the same condition
        stop the path at a free / hand-over event       (no leak on this path)
        at a return: report "A leaks here",
                     with the failing call that chose this path (trigger)
```

In plain words: start where memory is taken, walk every possible way out of the function, and complain about any way out that neither frees the memory nor gives it to someone else. The “skip” rules remove paths that cannot really happen.

4.2 Proving leaks (runtime verification)

```
mapping:  for n = 1, 2, 3, ...:
              load the module with fail-nth = n (only failures inside the module count)
              record which source line the n-th failure hit; stop when nothing fails
confirm:  for each candidate:
              pick n whose failure hits the candidate's trigger
              run 3 times with fail-nth = n; then kmemleak scan
verdict:  object from the candidate's line leaked  -> confirmed
          path ran, nothing leaked                  -> not reproduced
          trigger could not be failed               -> inconclusive
```

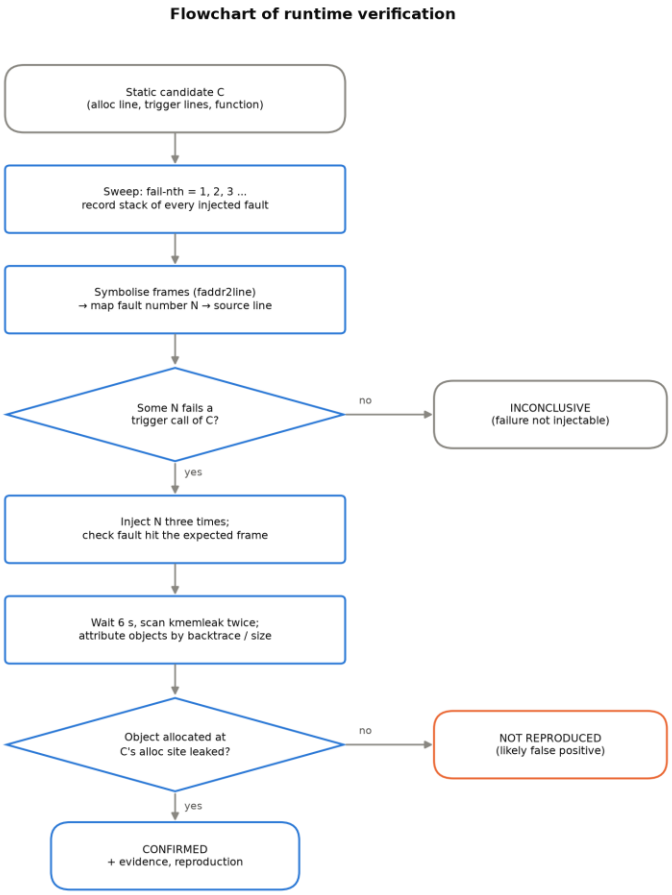


Figure 4.1: Flowchart of the runtime verification

5. Implementation Progress

By Review-I the core of the tool runs from start to finish on the benchmark: it compiles the modules, finds candidates, boots the kernel, injects the predicted failures and produces verdicts with kmemleak evidence. We estimate the project at about two thirds complete.

Table 5.1: Implementation status

Part	Status	Notes
Kernel build with kmemleak + fault injection	Done	Linux 6.18.55, clang
QEMU guest, init script, shared folder	Done	BusyBox, 9p share
klg-agent (sweep / repeat modes)	Done	fail-nth with module filter
klg_modwatch symbol logging	Done	for unloaded modules
Benchmark: 32 modules with ground truth	Done	17 leaks, 42 safe sites
Static analyser: aliases, paths, triggers	Done	C++/LLVM
Ownership hand-over, summaries, branch correlation	Done	reduces false alarms
Mapping and confirmation runs, verdicts	Done	end to end
Baselines (exhaustive, random, cppcheck)	In progress	Review-II
Fix patch generation and validation	In progress	Review-II

Dashboard	Prototype	findings page works
Real kernel modules	Planned	Review-II
Report, manual, presentation	In progress	Review-II

6. Screenshots of the Working Prototype

The screenshots below were taken from the running system.



Figure 6.1: The static analyser's report for one benchmark module

```
Ubuntu (WSL2) — python3 -m klg show
$ python3 -m klg show
klg_l01_two_alloc:14 initialize() kmalloc leaks at line 19 (-ENOMEM, high) -> confirmed
    leak at klg_l01_two_alloc.c:14 after failing initialize+0x46/0xf0
klg_l02_goto_ladder:32 sensor_setup() kmalloc leaks at line 54 (-ENOMEM, high) -> confirmed
    leak at klg_l02_goto_ladder.c:32 after failing sensor_setup+0x5f/0xb0
klg_l03_reg_fail:33 channel_open() kzalloc leaks at line 40 (error from hw_register_channel, high) -> confirmed
    leak at klg_l03_reg_fail.c:33 after failing channel_open+0x32/0x70
klg_l04_loop_partial:19 ring_alloc() kmalloc leaks at line 21 (-ENOMEM, high) -> confirmed
    leak in ring_alloc() after failing ring_alloc+0x48/0x2d0; frame has no line info (merged by optimisation), function's only allocation
    site is line 19
klg_l05_usercopy:20 cfg_store() kmalloc leaks at line 25 (-EFAULT, high) -> confirmed
    leak at klg_l05_usercopy.c:20 after failing cfg_store+0x42/0x90
klg_l06_wrapper_alloc:37 session_start() session_create leaks at line 43 (-ENOMEM, high) -> confirmed
    leak at klg_l06_wrapper_alloc.c:37 after failing session_start+0x31/0x80
klg_l07_wrapper_free:27 xfer_run() kzalloc leaks at line 33 (-ENOMEM, high) -> confirmed
    leak at klg_l07_wrapper_free.c:27 after failing xfer_run+0x43/0x90
klg_l08_kstrdup:16 profile_load() kstrdup leaks at line 22 (-ENOMEM, high) -> confirmed
    leak at klg_l08_kstrdup.c:16 after failing profile_load+0x3b/0x60
klg_l09_kcalloc:17 table_build() kmalloc_array leaks at line 34 (-ENOMEM, high) -> confirmed
    leak at klg_l09_kcalloc.c:17 after failing table_build+0x44/0xc0
klg_l10_vmalloc:22 fw_stage() vmalloc leaks at line 28 (-ENOMEM, high) -> confirmed
    leak at klg_l10_vmalloc.c:22 after failing fw_stage+0x37/0x80
klg_l11_kmem_cache:22 req_submit() kmem_cache_alloc leaks at line 28 (-ENOMEM, high) -> confirmed
    leak at klg_l11_kmem_cache.c:22 after failing req_submit+0x3d/0x90
klg_l12_multi_exit:14 packet_prepare() kmalloc leaks at line 27 (-ENOMEM, high) -> confirmed
    leak at klg_l12_multi_exit.c:14 after failing packet_prepare+0x6c/0x160
klg_l13_init_misc:42 ledctl_init() kzalloc leaks at line 49 (error from misc_register, medium) -> confirmed
    leak at klg_l13_init_misc.c:42 after failing init_module+0x3c/0xff0
klg_l14_cond_alloc:29 stats_setup() kzalloc leaks at line 39 (error from stats_calibrate, high) -> confirmed
    leak at klg_l14_cond_alloc.c:29 after failing stats_setup+0x4d/0x80
klg_l15_ida:21 widget_add() kzalloc leaks at line 27 (error from ida_alloc, high) -> inconclusive
    no fault point reaches the failing call (klg_l15_ida.c:25)
klg_l17_local_struct:21 transform_run() kmalloc leaks at line 27 (-ENOMEM, high) -> confirmed
    leak at klg_l17_local_struct.c:21 after failing transform_run+0x42/0x90
klg_s09_loop_cleanup:20 ring_alloc() kmalloc leaks at line 34 (-ENOMEM, high) -> not-reproduced
    predicted failure injected, no object left unreferenced
```

Figure 6.2: Findings and verdicts on the command line (python3 -m klg show)

```
[    7.575330] Demotion targets for Node 0: null
[    7.576091] kmemleak: Kernel memory leak detector initialized (mem pool avail
able: 15834)
[    7.602590] PM:   Magic number: 14:443:787
[    7.603612] netconsole: network logging started
[    7.607508] cfg80211: Loading compiled-in X.509 certificates for regulatory d
atabase
[    7.623428] kworker/u8:0 (59) used greatest stack depth: 14472 bytes left
[    7.643890] Loaded X.509 cert 'sforshee: 00b28ddf47aef9cea7'
[    7.646752] Loaded X.509 cert 'wens: 61c038651aabdcf94bd0ac7ff06c7248db18c600
'
[    7.651395] ALSA device list:
[    7.651860]   No soundcards found.
[    7.656666] faux_driver regulatory: Direct firmware load for regulatory.db fa
iled with error -2
[    7.658599] cfg80211: failed to load regulatory.db
[    8.509799] Freeing unused kernel image (initmem) memory: 3364K
[    8.511331] Write protecting the kernel read-only data: 28672k
[    8.516893] Freeing unused kernel image (text/rodata gap) memory: 440K
[    8.539851] Freeing unused kernel image (rodata/data gap) memory: 584K
[    8.831203] x86/mm: Checked W+X mappings: passed, no W+X pages found.
[    8.832496] Run /init as init process
[    9.151786] busybox (62) used greatest stack depth: 8 bytes left
KLG-GUEST-READY 6.18.55
_
```

Figure 6.3: Our Linux 6.18.55 kernel booting inside the QEMU virtual machine

```
klg-agent output inside the guest (fault injection)
$ klg-agent repeat --mode trigger --target /sys/kernel/debug/klg_klg_l03_reg_fail --nth 3 --count 3
@@BEGIN n=3
@@ATTEMPT n=3 ret=-1 errno=22 injected=1
K FAULT_INJECTION: forcing a failure.\x0aname fail_function, interval 1, probability 0, space 0, times -1
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? channel_open+0x32/0x70 [klg_l03_reg_fail]
K  ? klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
K  ? hw_register_channel+0x4/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x5/0x20 [klg_l03_reg_fail]
K  hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  channel_open+0x32/0x70 [klg_l03_reg_fail]
K  klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
@@END
@@BEGIN n=3
@@ATTEMPT n=3 ret=-1 errno=22 injected=1
K FAULT_INJECTION: forcing a failure.\x0aname fail_function, interval 1, probability 0, space 0, times -1
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? channel_open+0x32/0x70 [klg_l03_reg_fail]
K  ? klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
K  ? hw_register_channel+0x4/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x9/0x20 [klg_l03_reg_fail]
K  ? hw_register_channel+0x5/0x20 [klg_l03_reg_fail]
```

Figure 6.4: The agent injecting a failure inside the guest

```
times
verbose
verbose_ratelimit_burst
verbose_ratelimit_interval_ms

==== 2. Load and unload the real erofs module normally
# insmod /mnt/host/mods/erofs.ko && lsmod | grep '^erofs' && rmmod erofs
erofs                  126976  0
# grep -c 'unreferenced object' /sys/kernel/debug/kmemleak || true
0
(0 = kmemleak finds no leak)

==== 3. Fail exactly ONE allocation (fault 85) while erofs initialises
# klg-agent repeat --mode init --target /mnt/host/mods/erofs.ko --nth 85 --count
 1 --filter erofs --out /tmp/erofs.log | tail -2
--- the injected failure (kernel log):
[   43.680259] FAULT_INJECTION: forcing a failure.
[   43.683493]  ? __kmalloc_cache_noprof+0x41/0x420
[   43.683558]  ? sysfs_slab_add+0xa8/0x270
[   43.683577]  ? do_kmem_cache_create+0x560/0x650
[   43.683594]  ? __kmem_cache_create_args+0x12b/0x350
[   43.683766]  ? init_module+0x98/0xe0 [erofs]

==== 4. kmemleak: the slab cache was leaked
```

Figure 6.5: A failure injected while a real module (erofs) loads

```
cat /sys/kernel/debug/kmemleak (guest)
$ cat /sys/kernel/debug/kmemleak
unreferenced object 0xff11000005a88a18 (size 8):
  comm "klg-agent", pid 163, jiffies 4294752726
  hex dump (first 8 bytes):
    03 00 00 00 00 00 00 00                         ........
  backtrace (crc 8e8f07e3):
    __kmalloc_cache_noprof+0x1b5/0x420
    channel_open+0x1c/0x70 [klg_l03_reg_fail]
    klg_trigger_write+0x12/0x30 [klg_l03_reg_fail]
    full_proxy_write+0x5c/0xb0
    vfs_write+0x136/0x450
    ksys_write+0x79/0x100
    do_syscall_64+0xc8/0x260
    entry_SYSCALL_64_after_hwframe+0x77/0x7f
```

Figure 6.6: kmemleak reporting the leaked object, with its call stack

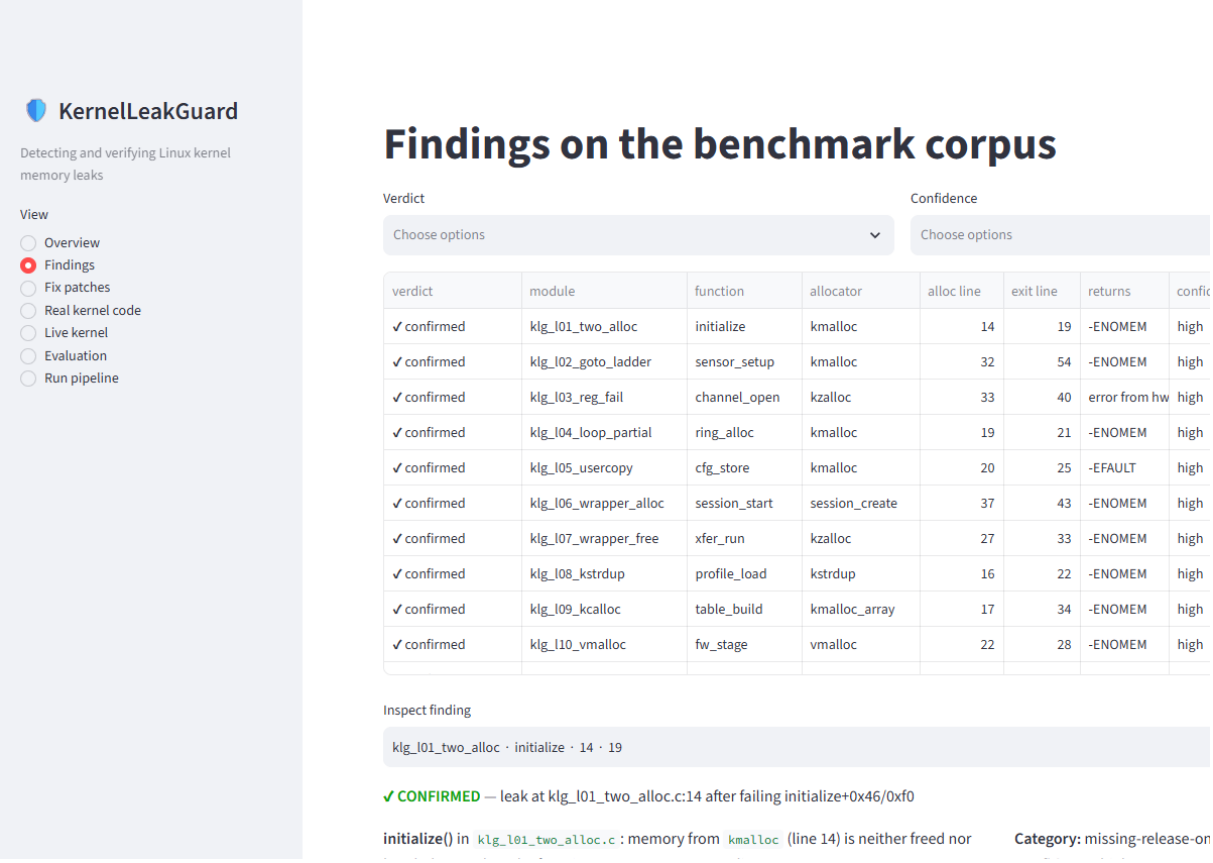


Figure 6.7: Dashboard prototype: the list of findings with verdicts

7. Challenges Faced

Not everything worked the first time. These were the hardest problems and how we solved them.

Table 7.1: Challenges and solutions

Challenge	What happened	How we solved it

Fault injection ignored	Failing a function by name did nothing for our calls.	The kernel's probe mechanism had to be built in its ftrace-based mode; with the other mode the injection never matched our task.
Leaks hidden from kmemleak	A real leak was found in some runs and missed in others.	Old copies of pointers on reused stacks and in reused memory made kmemleak think the memory was still used. We clear stacks and zero memory on allocation and free.
Module gone before the scan	When a module fails to load, Linux removes it before kmemleak reports, so stack traces showed only raw addresses.	We wrote klg_modwatch, a tiny module that logs every module's addresses and symbols.
Random injection crashed the guest	PID 1 also got failures and the kernel panicked.	Restrict injection to the agent's own task before turning it on.
Too many false alarms	Helpers that return error codes instead of NULL looked like leaks.	We model IS_ERR() like a NULL check and let runtime checks refute the rest.
Stack traces to source lines	The kernel's faddr2line script broke with clang builds.	We call it in the mode that works with LLVM builds.
Slow virtual machine	Windows security features blocked hardware virtualization (KVM) inside WSL2.	QEMU falls back to software emulation; the tool only fails the calls the analyser points at, which keeps runs short.

8. Plan for Review-II

- Run the baselines (fail every allocation, random failures, cppcheck) and compare.
- Generate fix patches and check them by compiling, re-analysing and re-testing.
- Finish the dashboard and write the user manual and final report.
- Try the tool on real Linux modules that need no hardware.
- Repeat the experiments several times to check that results are stable.

9. References

[1] Wang, W. “MLEE: Effective Detection of Memory Leaks on Early-Exit Paths in OS Kernels.” USENIX Annual Technical Conference (ATC), 2021, pp. 31–45.

[2] Linux kernel documentation: “Kernel Memory Leak Detector (kmemleak).” https://docs.kernel.org/dev-tools/kmemleak.html

[3] Linux kernel documentation: “Fault injection capabilities infrastructure.” https://docs.kernel.org/fault-injection/fault-injection.html

[4] Google. “syzkaller: an unsupervised coverage-guided kernel fuzzer.” https://github.com/google/syzkaller

[5] Emamdoost, N., Wu, Q., Lu, K., McCamant, S. “Detecting Kernel Memory Leaks in Specialized Modules with Ownership Reasoning (K-MELD).” NDSS, 2021.

[6] Lattner, C., Adve, V. “LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation.” CGO, 2004.

[7] Bellard, F. “QEMU, a Fast and Portable Dynamic Translator.” USENIX ATC, 2005.

[8] Cppcheck: a tool for static C/C++ code analysis. https://cppcheck.sourceforge.io/

[9] Silberschatz, A., Galvin, P. B., Gagne, G. Operating System Concepts, 10th ed. Wiley, 2018.

[10] Corbet, J., Rubini, A., Kroah-Hartman, G. Linux Device Drivers, 3rd ed. O’Reilly, 2005.